Week 2 - Domain 1: SDLC Automation - Deployment Patterns

Study week: Oct 26 - Nov 1, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Section	What it covers
10.1 - 10.2	How to compare patterns; mutable vs immutable infrastructure
10.3	All at once, in-place, rolling, rolling with additional batch, immutable, blue/green, canary, linear, traffic splitting, related patterns
10.4 - 10.6	Elastic Beanstalk policy table, patterns by AWS service, choosing a pattern
10.7 - 10.9	Exam traps, hands-on lab, check yourself

Part 10: Deployment Patterns

10.1 How to compare patterns

Questions give a requirement - "zero downtime", "keep full capacity", "cheapest", "fastest rollback", "test with real users first", "no mixed versions" - and ask for the right pattern and service. Compare on:

Question	What it means
Downtime?	Is the app unavailable at any point?
Capacity during the deployment?	Fewer servers while deploying?
Mixed versions?	Old and new serving users at the same time?
Rollback speed?	Instant (switch traffic back) or slow (redeploy old version)?
Cost?	Extra servers during the deployment?

10.2 Mutable vs immutable infrastructure

	Mutable (change existing servers)	Immutable (replace servers)
How	New version installed onto running servers	New servers with the new version; old ones removed
Patterns	In-place, all at once, rolling, rolling with additional batch	Immutable, blue/green, Auto Scaling instance refresh, new AMIs
Benefits	Cheaper, faster, no new servers	No configuration drift, clean known state, rollback by switching back, repeatable
Risks	Drift; a failed half-way update leaves a mess	More cost during deployment, slower to start new servers
Exam keywords	least cost, fastest	consistent, no drift, replace instead of patch, golden AMI

10.3 The patterns

All at once (in-place)

- New version deployed to **every server at the same time**, on existing servers.
- **Downtime: yes** (short). Capacity drops to **0%**. Rollback **slow** (redeploy). **Fastest and cheapest.**
- Use for dev / test or when downtime is acceptable.
- AWS: Beanstalk **All at once**; CodeDeploy `CodeDeployDefault.AllAtOnce` (in-place); `LambdaAllAtOnce`; `ECSAllAtOnce`.

In-place (general term)

- The new version replaces the old one **on the same servers**. All at once, rolling and rolling with additional batch are all in-place.

- In CodeDeploy, **in-place is a deployment type for EC2 / on-premises**; how many servers at once is set by the deployment configuration (AllAtOnce, HalfAtATime, OneAtATime, custom minimum healthy hosts, zonal).
- **On-premises servers can only use in-place.**

Rolling

- Servers updated in **batches** (e.g. 2 at a time). A batch is **taken out of the load balancer**, updated, put back when healthy, then the next batch.
- No downtime. Capacity **reduced by one batch** (10 servers, batch of 2 = 80%). **Mixed versions: yes.** Rollback **slow** (another rolling deployment). No extra cost.
- Risk: a failure halfway leaves some servers on new, some on old.
- AWS: Beanstalk **Rolling** (batch size fixed or %); CodeDeploy in-place HalfAtATime / OneAtATime / custom; **ECS rolling update** (`minimumHealthyPercent`, `maximumPercent`); **Kubernetes RollingUpdate** (`maxUnavailable`, `maxSurge`); **Auto Scaling instance refresh**; **CloudFormation AutoScalingRollingUpdate**.

Rolling with additional batch

- Like rolling, but **starts an extra batch of servers first** so you **keep full capacity** the whole time; the extra batch is removed at the end.
- No downtime. Capacity **100%**. Mixed versions: yes. Rollback slow. **Small extra cost** (one batch).
- Use for production that can't lose capacity but must stay cheaper than immutable / blue-green.
- AWS: Beanstalk **Rolling with additional batch**. ECS rolling with `maximumPercent` above 100 and `minimumHealthyPercent` 100 behaves the same (new tasks start before old ones stop).

Immutable

- A **whole new set of servers** is launched (Beanstalk: in a **temporary Auto Scaling group**); when healthy they go into service and the **old servers are terminated**.
- No downtime. Capacity 100% (briefly **double**). Mixed versions briefly. Rollback **fast and safe** - failed new servers are **terminated**; old servers were never changed. Cost: double servers briefly.
- Use for production needing the safest rollback, where a failed deployment must never affect running servers.
- AWS: Beanstalk **Immutable**; **CloudFormation AutoScalingReplacingUpdate (**`WillReplace: true`**)** - new group, old deleted only if the new one succeeds; **instance refresh** with new AMIs.

Blue/green

- Two **complete, separate environments**: **blue** (current) and **green** (new). Deploy and test green, **switch all traffic**, keep blue for rollback.
- No downtime. Capacity 100% (green full size). **Mixed versions: no** (all traffic switches at once). Rollback **fastest** (switch back). Cost **highest** (two environments).
- Use for big releases, fastest rollback, full testing of the new environment before users see it.

Switch method	How	Watch out
Route 53 DNS (weighted / simple)	Point the record at green or change weights 100/0 to 0/100	DNS caching (TTL) - some clients keep using blue; rollback not instant
ALB listener / target groups	Switch the listener to the green target group, or weighted target groups	Instant, no DNS caching - usually the best answer
Swap Auto Scaling groups behind one load balancer	Attach green, detach blue	No DNS change
Elastic Beanstalk swap environment URLs	Clone the environment, deploy, swap CNAMEs	DNS change (TTL); RDS must be outside both environments
CodeDeploy blue/green	EC2 (copy ASG + load balancer), ECS (two target groups), Lambda (alias)	Managed, with alarms and automatic rollback
API Gateway	Switch stage, or canary release on the stage	
CloudFront	Change origin, or continuous deployment (staging distribution)	

Blue/green things to plan for:

- **Databases:** usually shared - schema changes must be **backward compatible** (**expand and contract**: add new columns keeping old ones > deploy new code > remove old columns later).
- **Elastic Beanstalk:** an RDS database created **inside** the environment is **deleted when the environment is terminated** - create the database **separately** and connect with environment properties.
- **Sessions:** store **outside** the servers (**ElastiCache or DynamoDB**) or users are logged out on the switch.
- **Connection draining:** set the ALB **deregistration delay** so in-flight requests finish.

Canary

- Send a **small percentage** of real traffic to the new version (e.g. **10%**), **watch** metrics and alarms for a set time, then **all the rest at once** - or **roll back**.
- No downtime. Capacity 100%. Mixed versions on purpose during the test. Rollback fast. Cost low to medium.
- Use for testing with real users while limiting damage; automatic rollback on alarms.
- AWS: **CodeDeploy** `LambdaCanary10Percent5/10/15/30Minutes`, `ECSCanary10Percent5/15Minutes`, custom; **SAM** `DeploymentPreference: Canary...`; **API Gateway canary release** (canary % on a stage, then **promote**); **Lambda alias weighted routing**; **ALB weighted target groups**; **Route 53 weighted records**; **CloudFront continuous deployment**; **AppConfig** for configuration / flags.

Linear

- Traffic moves in **equal steps** over time (e.g. **10% more every minute**) to 100%, checking metrics at each step.
- **Canary = one small step, then everything at once. Linear = many equal steps** (more gradual, takes longer).
- Rollback fast.
- AWS: **CodeDeploy** `LambdaLinear10PercentEvery1Minute/2/3/10`, `ECSLinear10PercentEvery1Minutes/3Minutes`, custom; **AppConfig** Linear and **Exponential** strategies (bake time, alarm rollback).

Traffic splitting (Elastic Beanstalk)

- Beanstalk launches a **full set of new servers** (like immutable) in a **temporary Auto Scaling group**, then **sends a percentage of real traffic** to them for an **evaluation time**. Healthy = all traffic moves and old servers terminate; unhealthy = traffic moves back and new servers terminate.
- **Canary testing built into Beanstalk**, using an **ALB**. Settings: **traffic split %** and **evaluation time**.
- No downtime. Capacity 100%. Rollback quick and automatic. Cost: double servers briefly.
- Exam keyword: "Beanstalk ... 10% of traffic to the new version for 15 minutes, then everything" = **traffic splitting**.

Related patterns:

Pattern	What it is	AWS
Feature flags / dark launch	Code deployed but switched off, turned on for chosen users - deploy and release are separate steps	AWS AppConfig feature flags
A/B testing	Two versions to different users to compare business results	AppConfig, ALB weighted target groups, CloudFront (CloudWatch Evidently retired - use AppConfig)
Shadow / mirror testing	Real traffic copied to the new version, responses discarded	Custom proxies / service meshes (rare on the exam)
Recreate	Stop all old, then start new - downtime	Kubernetes Recreate strategy

10.4 Elastic Beanstalk deployment policies

Policy	Downtime	Failed deployment impact	Deploy time	Capacity	Rollback	Code goes to
All at once	Yes	Downtime	Fastest	Drops to 0	Redeploy manually	Existing servers
Rolling	No	One batch out of service; mixed versions	Fast	Reduced by one batch	Redeploy manually	Existing servers
Rolling with additional batch	No	Minimal if first batch fails, otherwise like rolling	Medium	Full	Redeploy manually	New and existing servers
Immutable	No	Minimal (new servers terminated)	Slow	Full, briefly double	Terminate new servers	New servers
Traffic splitting	No	Only the split traffic, briefly	Slow	Full, briefly double	Reroute traffic, terminate new servers	New servers
Blue/green (swap URLs)	No	Minimal	Slow	Full (second environment)	Swap URLs back	New environment (DNS change)

- **Rolling settings:** batch size (fixed or %), healthy threshold (OK / Warning / Degraded / Severe), **ignore health check** (risky), command timeout.
- **Blue/green is not a deployment policy** - clone the environment and **Swap environment URLs**, or use Route 53.
- **Configuration changes** (e.g. new instance type) can also replace servers; **rolling updates** (time-based or health-based) control that. **Deployment policies** control **application version** deployments - two separate settings.

10.5 Patterns by AWS service

Service	Patterns and settings
Elastic Beanstalk	All at once, Rolling, Rolling with additional batch, Immutable, Traffic splitting, blue/green (swap URLs)
CodeDeploy EC2 / on-premises	In-place (AllAtOnce, HalfAtATime, OneAtATime, custom minimum healthy hosts, zonal); blue/green (EC2 only, needs a load balancer)
CodeDeploy Lambda	Canary, Linear, AllAtOnce (alias shifts traffic)
CodeDeploy ECS	Blue/green with Canary, Linear or AllAtOnce shifting (two target groups)
ECS native deployments	Rolling update (minimumHealthyPercent, maximumPercent) + deployment circuit breaker (automatic rollback when tasks keep failing) + alarm-based rollback; newer native blue/green, linear, canary (check docs)
EKS / Kubernetes	RollingUpdate (maxSurge, maxUnavailable), Recreate; blue/green or canary with Argo Rollouts / Flagger or ALB ingress weighted target groups
Auto Scaling groups	Instance refresh: rolling replacement with MinHealthyPercentage / MaxHealthyPercentage, checkpoints, skip matching, automatic rollback (alarms / failed health)
CloudFormation UpdatePolicy	AutoScalingRollingUpdate (MinInstancesInService, MaxBatchSize, PauseTime, WaitOnResourceSignals) = rolling; AutoScalingReplacingUpdate (WillReplace) = immutable
Lambda (no CodeDeploy)	Versions + aliases with weighted routing (e.g. 90% v1 / 10% v2), set manually
API Gateway	Canary release on a stage (canary %, canary stage variables), then promote
Route 53	Weighted records (canary / blue-green), failover records

Service	Patterns and settings
ALB	Weighted target groups in a forward action (up to 5 target groups, optional group stickiness) - instant switch
CloudFront	Continuous deployment: staging distribution gets a % of traffic or traffic with a header, then promote
AppConfig (config / feature flags)	Linear, Exponential, AllAtOnce; bake time; automatic rollback on CloudWatch alarms

10.6 Choosing the right pattern

Requirement	Pattern
Cheapest and fastest, downtime OK (dev / test)	All at once
No downtime, lower capacity OK, no extra cost	Rolling
No downtime and full capacity, lowest extra cost	Rolling with additional batch
Failed deployment must not affect running servers; safe quick rollback	Immutable
Fastest rollback, no mixed versions, test whole new environment	Blue/green
Small share of real users first, then everything	Canary (Beanstalk: traffic splitting)
Increase traffic evenly in steps	Linear
Turn features on / off without deploying	Feature flags (AppConfig)
Instant switch without DNS caching problems	ALB target groups (weighted), not Route 53
On-premises servers	In-place only (CodeDeploy)
Replace every EC2 instance with a new AMI gradually	Auto Scaling instance refresh (or CloudFormation rolling / replacing update)
ECS tasks keep failing to start - roll back automatically	ECS deployment circuit breaker

10.7 Exam traps: quick table

Scenario	Answer
Beanstalk production must keep full capacity, cheaply	Rolling with additional batch
Beanstalk: failed deployment must not touch existing servers	Immutable
Beanstalk: 10% of traffic for 15 minutes, then everything	Traffic splitting
Beanstalk blue/green	Clone environment + Swap environment URLs (or Route 53)
Beanstalk blue/green deleted the database	RDS was inside the environment - create it separately
Users logged out after the blue/green switch	Sessions on servers - use ElastiCache or DynamoDB
Some users still reach blue after a DNS switch	DNS TTL / caching - use an ALB switch, or lower TTL ahead of time
Database schema change during blue/green	Backward-compatible changes (expand and contract)
Lambda: 10%, wait, then the rest	Canary (LambdaCanary10Percent...)
Lambda: another 10% every few minutes	Linear (LambdaLinear10PercentEvery...)
Gradual Lambda shift without CodeDeploy	Alias weighted routing
Canary for a REST API stage	API Gateway canary release, then promote
Canary a new CloudFront configuration	CloudFront continuous deployment

Scenario	Answer
ECS rolling deploy stuck because tasks keep crashing	Deployment circuit breaker with rollback
Replace ASG instances with a new AMI, keep 90% healthy	Instance refresh MinHealthyPercentage 90 (or AutoScalingRollingUpdate)
CloudFormation: whole new ASG, keep old one if it fails	AutoScalingReplacingUpdate WillReplace true
Lowest cost, downtime acceptable	All at once
Mixed versions not allowed at any time	Blue/green (switch all at once), not rolling / canary
Roll out a configuration change gradually with alarm rollback	AppConfig linear / exponential strategy + alarms

10.8 Hands-on lab (about 2 hours)

1. **Elastic Beanstalk:** environment with 4 instances behind an ALB; deploy v2 with **Rolling** (batch 1), **Rolling with additional batch**, **Immutable**, **Traffic splitting** (20% for 5 minutes); watch instance count and health.
2. **Beanstalk blue/green:** clone the environment, deploy v3 to the clone, **Swap environment URLs**; check the CNAME with `nslookup`.
3. **ALB weighted target groups:** two target groups, forward action 90/10, then 0/100 for an instant switch.
4. **Lambda alias weighted routing:** publish v1 and v2; alias v1 90% / v2 10%; invoke many times and count versions.
5. **API Gateway canary:** canary on the `prod` stage at 10%, deploy, then **promote**.
6. **Instance refresh:** change the launch template AMI; refresh with MinHealthyPercentage 75 and a checkpoint at 50%.
7. Clean up: Beanstalk environments, extra target groups, everything else.

10.9 Check yourself

1. Which Beanstalk policy keeps full capacity at the lowest extra cost?
2. Difference between canary and linear?
3. Why can a DNS-based blue/green switch take a while, and what's better?
4. What must you do with the RDS database to use blue/green with Beanstalk?
5. Which patterns change existing servers, and which replace them?
6. How do you send 10% of a Lambda function's traffic to a new version without CodeDeploy?

Answers:

1. **Rolling with additional batch.**
2. **Canary:** one small step, wait, then **all the rest**. **Linear: equal steps** over time.
3. **DNS caching (TTL)** keeps some clients on the old address; **ALB target group switching** (incl. weighted) is instant.
4. Create the **RDS database outside the Beanstalk environment**; both environments connect to it.
5. **Change existing:** all at once, rolling, rolling with additional batch (CodeDeploy in-place). **Replace:** immutable, blue/green, traffic splitting, instance refresh with a new AMI.
6. Publish a new **version** and set the **alias weighted routing** (e.g. 90% old / 10% new).